Pearls AQI Predictor — Project Summary

Bahawalpur, Pakistan · Data Science Internship @ 10Pearls Pakistan

---

1. Overview

Pearls AQI Predictor is an end-to-end, fully automated machine learning system that forecasts Air Quality Index (AQI) up to three days ahead for Bahawalpur, Pakistan — a desert city with two distinct pollution regimes: winter smog and summer dust storms.

The project follows a 100% serverless architecture: live hourly data collection, a feature store, automated daily model retraining, and a public-facing interactive dashboard with model explainability — built entirely on free-tier tools.

Objective (per official spec): predict AQI for the next 3 days using automated data collection, feature engineering, model training, and real-time predictions through a web dashboard.

2. System Architecture

```
┌──────────────────┐      ┌──────────────────────┐
│ OpenWeather API  │      │   Open-Meteo API     │
│ Live pollution   │      │ Historical backfill  │
│ (hourly reads)   │      │ (one-time, 2 yrs)    │
└────────┬─────────┘      └──────────┬───────────┘
         │ hourly                    │ one-time
         ▼                           │ bulk load
┌──────────────────┐                 │
│ GitHub Actions   │                 │
│ live_pipeline.py │                 │
│ cron: every hour │                 │
└────────┬─────────┘                 │
         │ insert (typed)            │
         ▼                           ▼
┌──────────────────────────────────────────┐
│              HOPSWORKS                   │
│       Feature Store (Serverless)         │
│    bahawalpur_aqi_features_v2            │
└────────┬─────────────────────┬───────────┘
         │                     │
read (daily, full)     read (on dashboard load)
         │                     │
         ▼                     ▼
┌──────────────────┐  ┌──────────────────────┐
│ GitHub Actions   │  │   Streamlit App      │
│ train_model.py   │  │     (app.py)         │
│ cron: 2 AM PKT   │  │                      │
│                  │  │ Live gauge, 24h trend,│
│ • Feature eng.   │  │ 3-day forecast cards,│
│ • Train RF+Ridge │  │ hazard alerts        │
│ • 3 horizons     │  └──────────┬───────────┘
│   (24/48/72h)    │             │ download models
└────────┬─────────┘             │
         │ register              │
         ▼                       │
┌──────────────────┐             │
│   HOPSWORKS      │◄────────────┘
│  Model Registry  │
│ ridge_day1/2/3   │
│ (versioned)      │
└────────┬─────────┘
         │
         ▼
┌──────────────────────┐
│      SHAP Layer      │
│ LinearExplainer per  │
│ model → waterfall chart│
│ "Why this prediction?"│
└──────────────────────┘
```

Flow in plain words: 1. Every hour, GitHub Actions fetches live pollution data and writes it into Hopsworks. 2. Once a day, a separate GitHub Actions job reads the full dataset, retrains 3 Ridge models (one per forecast horizon), and registers them with real accuracy metrics. 3. The Streamlit dashboard reads the latest data and models on load, generates a live 3-day forecast, and uses SHAP to explain exactly why each prediction came out the way it did. 4. The historical backfill (Open-Meteo) only ran once, to seed 2 years of training history before the hourly pipeline took over as the ongoing source of truth.

3. Tech Stack

Component	Choice	Why
Live data	OpenWeather Air Pollution API	Free tier, bundles weather + pollution, satellite-based (works for smaller cities where AQICN has no ground stations)
Historical backfill	Open-Meteo	Genuinely free, no API key, real historical endpoint
Feature Store / Model Registry	Hopsworks (serverless)	Free tier needs no credit card, unlike Vertex AI
Models	Random Forest + Ridge Regression	Compared on real metrics; Ridge won on all 3 forecast horizons
Explainability	SHAP (`LinearExplainer`)	Exact, deterministic explanations for a linear model
Dashboard	Streamlit + Plotly	Fastest to build a multi-section interactive dashboard
Automation	GitHub Actions	Free, spec-approved alternative to Apache Airflow
Deployment	Streamlit Community Cloud	Free hosting built for Streamlit apps

4. Data Pipeline

AQI calculation uses the EPA breakpoint interpolation formula, implemented from scratch (rather than a third-party library) for full understanding and defensibility:

```
AQI = ((AQI_high − AQI_low) / (Conc_high − Conc_low)) × (Conc − Conc_low) + AQI_low
```

Extreme values (e.g. a real dust storm event with PM10 readings of 685–704) are capped at AQI 500 rather than dropped — matching how real-world AQI systems like AirNow and IQAir handle hazardous extremes.

Feature engineering: - Time features: `hour`, `day_of_week`, `month` - Derived features: `aqi_24h_ago`, `aqi_change_rate`, `aqi_rolling_avg_24h` — added after the first model attempt scored a negative R^2, since raw snapshot data alone gave the model no sense of trend or momentum

Targets: three separate forecasts — `target_day1` (+24h), `target_day2` (+48h), `target_day3` (+72h) — rather than one single 72-hour number, since day-by-day forecasts are both more useful and more learnable.

Train/test split: chronological (oldest 80% train, newest 20% test), never randomly shuffled — critical for time-series data to avoid letting the model "see" the future during evaluation.

5. Model Results

Horizon	Random Forest R^2	Ridge R^2	Ridge RMSE
Day 1 (24h)	0.25	**0.31**	34.13
Day 2 (48h)	0.15	**0.19**	36.80
Day 3 (72h)	0.01	**0.15**	37.67

Ridge Regression won on every horizon. Accuracy naturally decreases as the forecast window extends — a realistic pattern that mirrors real weather forecasting, where near-term predictions are always more reliable than longer-range ones.

6. Explainability (SHAP)

Each Ridge model is paired with a `shap.LinearExplainer`, built on the full training background set (no subsampling, fully deterministic). The dashboard shows a waterfall chart for the live Day-1 prediction, breaking down exactly how each feature pushed the forecast up or down from the model's average baseline.

Key findings: - **Short-term (Day 1):** dominated by today's actual pollutant readings (`aqi_pm25`, `pm10`) — pollution has strong short-term persistence. - **Long-term (Day 3):** `aqi_pm10` (coarse dust) becomes the dominant driver, consistent with Bahawalpur's desert dust-storm dynamics mattering more at longer horizons than short-term smoke/smog.

7. Real Problems Solved

A selection of genuine engineering challenges encountered and fixed during the project:

- **Negative R^2 (−0.28) on first model attempt** — diagnosed as a seasonal distribution-shift problem (training on winter/spring, testing on summer dust-storm season); fixed by extending backfill to 2 full years and adding momentum-based derived features.
- **PM10 breakpoint table had gaps** at bracket boundaries, causing `None` values and crashes on real decimal-precision data; fixed by correcting to EPA's actual decimal boundary convention.
- **GitHub Actions runners are ephemeral** — an architectural insight caught before it caused data loss: local file saves would be silently lost every hour, so the live pipeline writes directly to Hopsworks instead.
- **Hopsworks schema type mismatch in production** — pandas inferred `int64` instead of `float64` for `pm2_5` the first time a whole-number reading appeared, breaking both automated pipelines; fixed with explicit `.astype()` casting before every insert, with a broader lesson about never trusting upstream data types at a system boundary.
- **Streamlit dashboard styling arc** — resolved caching issues (`@st.cache_resource` for stable Hopsworks connections), chart/card layout conflicts, and loading-spinner centering, arriving at a clean dark-themed, fully responsive dashboard.
- **Deployment dependency resolution failure** — Streamlit Cloud picked an incompatible `hopsworks` version due to an unpinned `requirements.txt`; fixed by pinning exact working versions and restoring a dependency (`hops-deltalake`) that had been accidentally trimmed.

8. Known, Accepted Limitations

- GitHub Actions free-tier scheduled triggers are not precise — observed real gaps of 40 minutes to 4+ hours between hourly runs. This is a documented platform limitation, not a bug.
- `fg.read()` (reading the full feature store) takes 7–18 seconds — mitigated with caching, not eliminated.
- $CO/NO_2/O_3/SO_2$ are collected but not used in AQI calculation, due to a genuine unit mismatch (OpenWeather gives µg/m³; EPA's gas breakpoint tables need ppm/ppb) — flagged as a future enhancement, not guessed around.

9. Future Enhancements (not yet built)

- Multi-city support (schema already includes a `city` column, currently constant, specifically to make this easier later)
- 6-pollutant grid + weather "Current Conditions" card (temperature, humidity, pressure)
- Deep learning model (LSTM) — explicitly optional per spec; current 2-model comparison already satisfies the "various models" requirement, and Ridge already generalizes well on this feature set

10. Author

Abdul Wahab Data Science Intern @ 10Pearls Pakistan

Repository: `Wahab-300/aqi-predictor-10Pearls`